

AUFGABE ZUR VORLESUNG

Übung 02 – Bind Mounts und Volumes

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema Docker: Code hineinreichen, Daten sichern
Dozent Dr. Ulrich Winkler
Semester 5. Semester · Informationstechnik

Vorlesung: Deck *docker-data* **Ziel:** Zwei Wege verstehen, wie Daten in und aus einem Container gelangen: den **Bind Mount** (ein Host-Ordner erscheint im Container – ideal für Code) und das **Volume** (von Docker verwalteter Speicher – ideal für Daten, die bleiben sollen).

Kontext

In diesem Ordner liegt `log_tool.py` – ein winziges Logbuch. Es schreibt Einträge in die Datei `$LOG_FILE` (Voreinstellung `/data/log.txt`) und kann sie auflisten:

```
python log_tool.py add "Docker ist toll"
python log_tool.py list
```

Sie führen dieses Skript in einem Container aus, **ohne** ein eigenes Image zu bauen: Der **Code** kommt per Bind Mount hinein, die **Daten** landen in einem Volume. Beobachten Sie das Volume anschließend in Portainer.

Aufgabenstellung

1. Führen Sie `log_tool.py` in einem `python`-Container aus, indem Sie den **aktuellen Ordner per Bind Mount** nach `/app` mounten. Fügen Sie zwei Einträge hinzu und lassen Sie sie auflisten.
2. Sie werden feststellen: Nach `docker run --rm` ist die Logdatei weg. Warum?
3. Legen Sie ein **Volume** `logdaten` an und mounten Sie es nach `/data`. Fügen Sie Einträge hinzu, löschen Sie den Container und starten Sie einen **neuen** – die Einträge sind noch da.
4. Ändern Sie `log_tool.py` auf dem Host (z. B. den Text bei „gespeichert:“). Führen Sie den Container erneut aus – die Änderung wirkt **sofort**, ohne Neubau. Erklären Sie, warum (Bind Mount).
5. Sehen Sie sich das Volume in Portainer an (**Volumes**) und mit `docker volume inspect logdaten`.

Tipp: Zwei Mounts gleichzeitig sind erlaubt – einer für den Code (Bind Mount), einer für die Daten (Volume).

Musterlösung

```
cd uebungen/02_bind_mounts_volumes

# 1) Code per Bind Mount hinein, im Container ausführen
```

```
docker run --rm -v "$PWD":/app -w /app python:3.13-slim \
    python log_tool.py add "Erster Eintrag"
docker run --rm -v "$PWD":/app -w /app python:3.13-slim \
    python log_tool.py add "Zweiter Eintrag"
docker run --rm -v "$PWD":/app -w /app python:3.13-slim \
    python log_tool.py list
# -> (noch keine Einträge)? Genau: siehe Punkt 2.

# 2) Ohne Volume liegt /data/log.txt IM Container. Mit --rm wird der Container
#     nach jedem Lauf gelöscht -> die Datei ist jedes Mal weg.

# 3) Volume für die Daten anlegen und einhängen
docker volume create logdaten
docker run --rm -v "$PWD":/app -w /app -v logdaten:/data python:3.13-slim \
    python log_tool.py add "Bleibt erhalten"
docker run --rm -v "$PWD":/app -w /app -v logdaten:/data python:3.13-slim \
    python log_tool.py add "Und noch einer"
# neuer Container, gleiches Volume -> Einträge sind noch da:
docker run --rm -v "$PWD":/app -w /app -v logdaten:/data python:3.13-slim \
    python log_tool.py list

# 4) log_tool.py auf dem Host bearbeiten (z.B. "gespeichert:" -> "OK,
#     gemerkt:"),
#     dann erneut ausführen -> Änderung ist sofort aktiv, weil der Bind Mount
#     den Host-Ordner live in den Container spiegelt (kein Image-Neubau nötig).

# 5) Volume inspizieren
docker volume inspect logdaten
```

Was Sie gelernt haben: Bind Mount = Host-Ordner live im Container (perfekt für Code während der Entwicklung); Volume = von Docker verwalteter, persistenter Speicher (perfekt für Daten). Ohne persistenten Speicher sind Container-Daten flüchtig.